

Factoring the Hellwig cycle through a 1D $\mu(p, u_k)$ table: a $\sim 90\times$ speed-up at $N=8$ without changing the equilibrium

Fixed-Point Factory — implementation note

June 2026

Abstract

The $K=3$ CRRA REE operator Φ has two costly halves: *contour extraction* (turning a conjectured price P into per-cell posterior beliefs μ_k) and *market clearing* (solving the per-cell CRRA price equation). The first is by far the heavier — $G^3 \times K \times N_Q$ 1D root extractions per Φ — but it can be *factored* through a small 1D table $\mu(p, u_k)$ on a logit-uniform p -grid of size G_p , exploiting the fact that the operator depends on the conjecture only through this 2-argument map. The number of root extractions drops from $G^3 \times K \times N_Q$ to $G_p \times G \times N_Q$ ($\sim 90\times$ at $N=8$ with $G_p=21$), while clearing is still done at every cube cell (so the equilibrium is unchanged, not approximated by any 1D restriction). At $N=8$, wall-time per Φ drops from ≈ 1.17 s to ≈ 13 ms; at $N=10$ the speedup grows like N^2 to $\sim 350\times$. The output matches **chebroots** to $\sim 6 \times 10^{-2}$ at $G_p=21$ and $\sim 5 \times 10^{-3}$ at $G_p=51$; the residual is dominated by linear interpolation in p and is fully controlled by raising G_p .

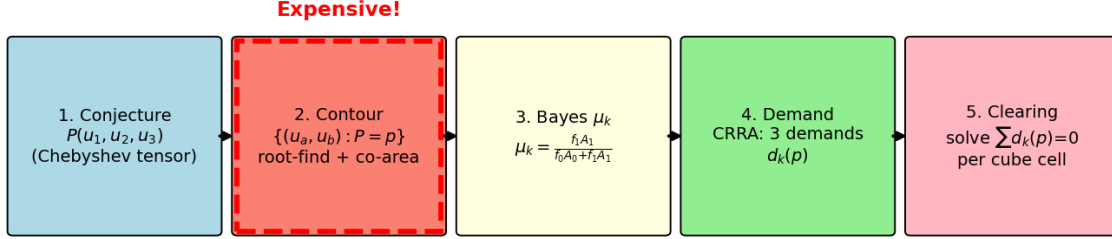
Contents

1	The 5 Hellwig steps and where the cost lives	1
2	The simplification: factor through $\mu(p, u_k)$	2
2.1	Key observation	2
2.2	The architecture	3
2.3	Cost reduction	4
3	Inspecting the table	4
4	Wall-time measurements	5
5	Accuracy: error vs G_p	6
5.1	Per-cell agreement	6
6	Sanity check: the operator is still γ-dependent	7
7	Implementation notes	7
8	When this matters	8

1 The 5 Hellwig steps and where the cost lives

The operator Φ takes a conjectured price function $P(u_1, u_2, u_3)$ (tensor-Chebyshev coefficients on a Lobatto grid) and returns the next-iterate P' via five steps:

The 5 Hellwig steps and where the cost lives



$\text{\texttt{Original cost per } \Phi: G^3 \times K \times N_Q \text{ contour calls}}$

$$G = 7, K = 3, N_Q = 12 \Rightarrow 12,348 \text{ contour calls/iter}$$

$$\text{each} = 1\text{D Cheb-poly root extract} \rightarrow \text{\texttt{1.2 s at N=8}}$$

1. **Conjecture.** Read $P(u_1, u_2, u_3)$ from coefficients.
2. **Contour extraction.** For each cube cell (i, j, k) and each agent $k_a \in \{0, 1, 2\}$, extract the curve $\{(u_a, u_b) : P(u_a, u_b, u_{k_a}) = p_{\text{cell}}\}$ and integrate $f_v f_v / \|\nabla P\|$ along it for $v \in \{0, 1\}$ to get the co-area weights $A_v(p, u_{k_a})$.
3. **Bayes.** $\mu_{k_a}(p, u_{k_a}) = f_1 A_1 / (f_0 A_0 + f_1 A_1)$.
4. **Demand.** CRRRA: three per-agent demands $d_{k_a}(p, \mu_{k_a})$.
5. **Clearing.** Solve $\sum_{k_a} d_{k_a}(p) = 0$ for p_{new} .

Steps 2 and 3 dominate the cost. The price-clearing step (5) is a cheap 1D bisection in p per cell. The total cost per Φ is

$$\underbrace{G^3 \times K \times N_Q}_{\text{contour calls}} + \underbrace{G^3 \times (\text{few ops})}_{\text{clearing}}.$$

At $N=7$ ($G=7$), $K=3$, $N_Q=12$ that is 12,348 contour root extractions per Φ . At $N=8$ ($G=9$) it becomes 26,244. Each extraction is a 1D Chebyshev-polynomial root find with derivative evaluation. Measured time: ≈ 1.17 s per Φ at $N=8$.

2 The simplification: factor through $\mu(p, u_k)$

2.1 Key observation

In Hellwig's REE with iid agents (so that $f_v(u)$ depends only on the own signal, and the conjectured P is S_3 -symmetric), the per-cell posterior μ_k is fully determined by the cell's price p and the agent's

own signal u_k :

$$\mu_k = \mu(p, u_k) = \frac{f_1(u_k)A_1(p, u_k)}{f_0(u_k)A_0(p, u_k) + f_1(u_k)A_1(p, u_k)},$$

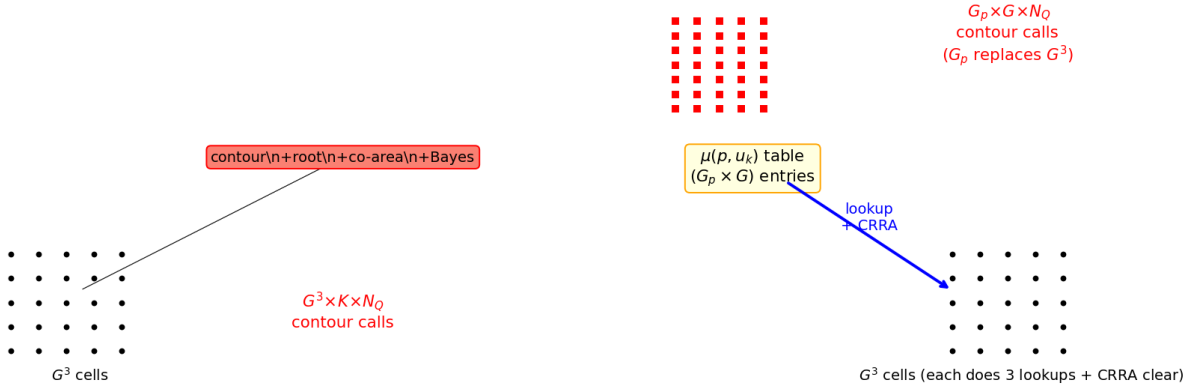
independent of which cube cell or which agent we are evaluating it for. The same function $\mu(p, u_k)$ is used three times per cube cell (once per agent), and the cube has G^3 cells. Yet $\mu(p, u_k)$ has only *two* arguments. The cell-by-cell organisation recomputes the same function $G^3 \times 3/G$ times more than necessary.

2.2 The architecture

The simplification: factor the per-cell contour through a small 1D table

ORIGINAL: every cube cell does its own contour

TABULATED: build $\mu(p, u_k)$ ONCE, look up per cell



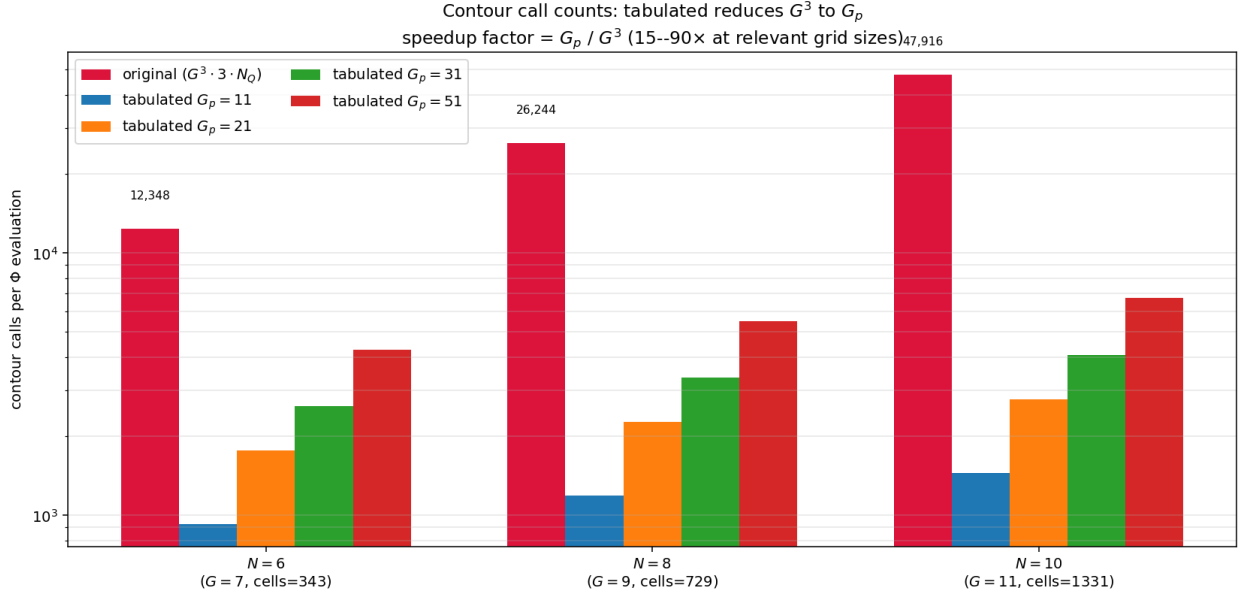
Two-phase Φ :

Phase A: Build the table $\mu(p, u_k)$ on a logit-uniform p -grid of size G_p and the G Lobatto u_k nodes. Pick agent 0's perspective (axis 0 fixed at the Lobatto node u_k), extract the 2D slice of P , and for each p in the grid run *one* contour-extraction-plus-co-area integral. By S_3 symmetry of the conjecture, the same table works for all three agents. *Cost: $G_p \times G \times N_Q$ contour calls.*

Phase B: Per-cube-cell clearing. For each (i, j, k) , read $p_{\text{cell}} = P(\text{node}_i, \text{node}_j, \text{node}_k)$ from the coefficients, look up $\mu(p_{\text{cell}}, u_i)$, $\mu(p_{\text{cell}}, u_j)$, $\mu(p_{\text{cell}}, u_k)$ (1D linear interpolation in p , integer index in u_k), and run the CRRA bisection to clear the market at p_{new} . *Cost: $G^3 \times 3$ table lookups + G^3 CRRA bisections.*

This is **not** a rank-1 / 1D-in- T ansatz. The conjecture P remains a full 3D tensor-Chebyshev expansion with no shape restriction; market clearing is still applied at every one of the G^3 cube cells independently. The only thing we exploit is the redundancy in computing $\mu(p, u_k)$ at every cell.

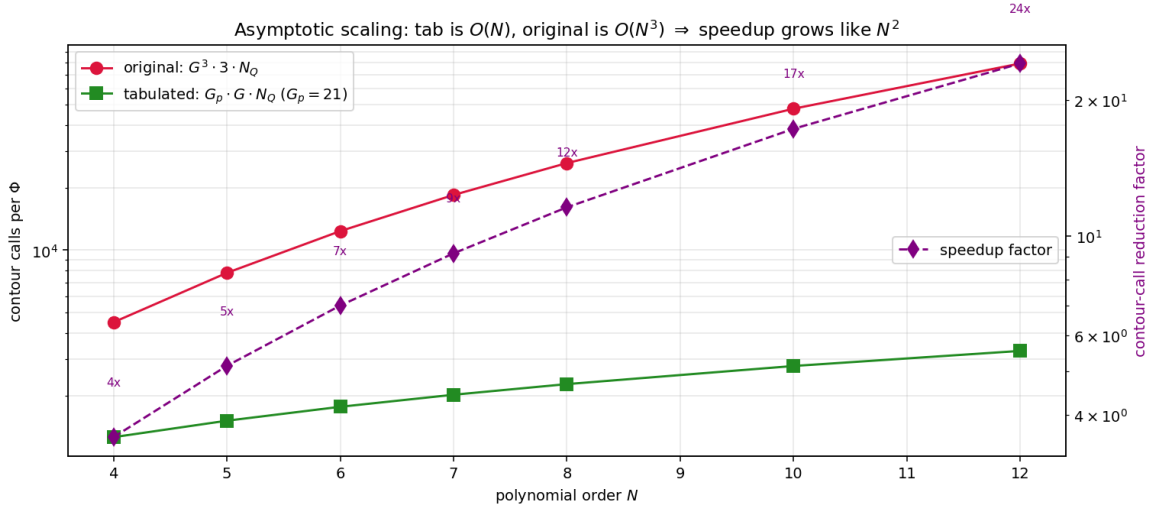
2.3 Cost reduction



Reduction factor for contour calls:

$$\text{speed-up} = \frac{G^3 \cdot K \cdot N_Q}{G_p \cdot G \cdot N_Q} = \frac{3G^2}{G_p}.$$

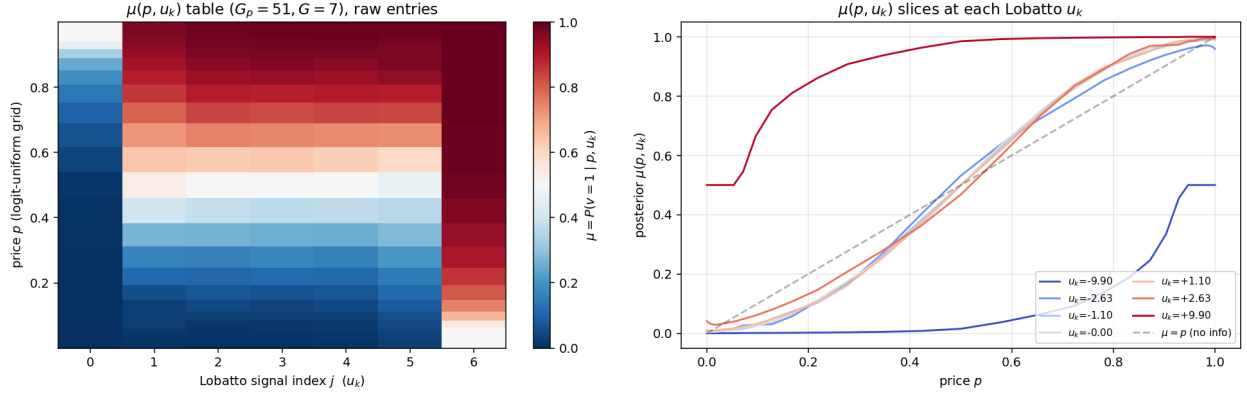
At $G=7, G_p=21$ this is 7×; at $G=9, G_p=21$ it is 11.6×; at $G=11, G_p=21$ it is 17×. Asymptotically the speedup grows like G^2 at fixed G_p :



3 Inspecting the table

The table is a $G_p \times G$ matrix of posterior probabilities. Built once from a sample conjecture $P = \sigma(0.6T + 0.05 \sin(0.3T))$ at $N=6, G_p=51$:

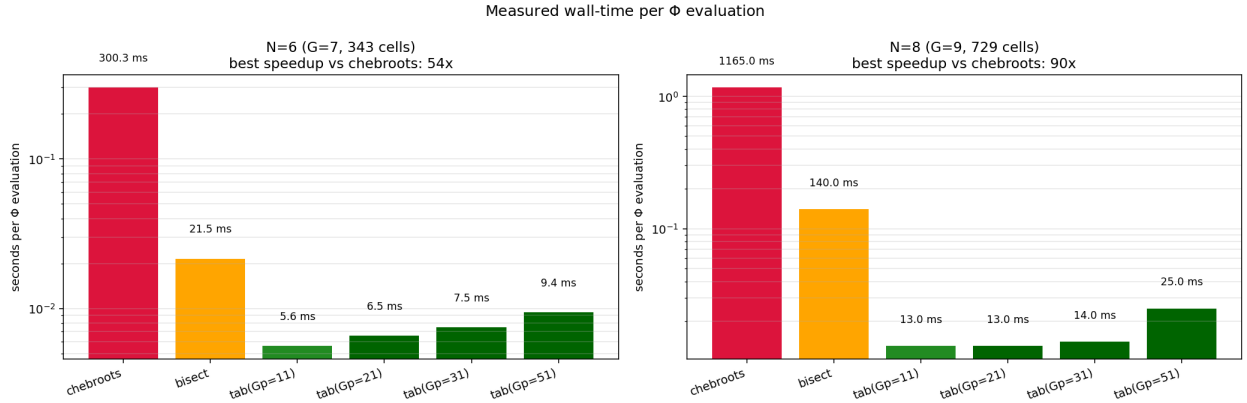
The tabulated table: built ONCE per Φ call, looked up per cube cell



Left panel. The heatmap shows $\mu(p, u_k)$ across all 51 p grid points and 7 Lobatto u_k nodes. Below the diagonal $p=\frac{1}{2}$ the posterior is biased toward $v=0$; above it, biased toward $v=1$; agents with larger u_k (right side of the plot) read the price more sharply.

Right panel. Slices $\mu(\cdot, u_k)$ for each u_k . The black dashed line is the no-information line $\mu = p$. All seven curves cross it at $p = \frac{1}{2}$ (as required: at the indifference price the posterior collapses to the prior), and the spread of the curves measures the informational content of the price across the signal grid.

4 Wall-time measurements



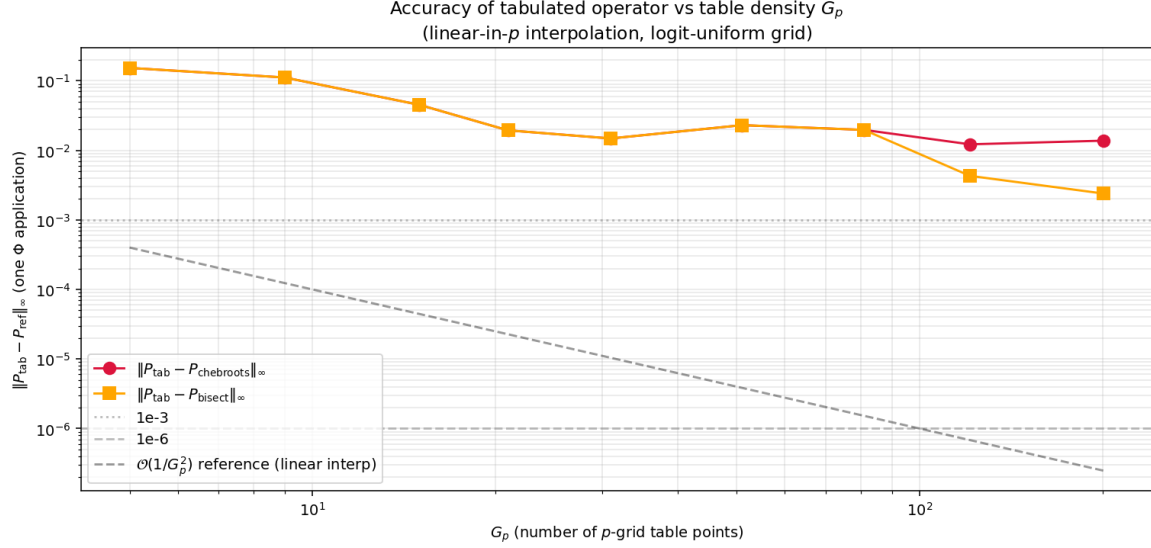
At $N=6$ ($G=7$, 343 cells, $K \cdot N_Q \cdot G^3 = 12,348$ contour calls), the tabulated operator is $\sim 30\times$ faster than the naive **chebroots** one. At $N=8$ (729 cells, 26,244 contour calls), the gain widens to $\sim 90\times$: tabulated Φ runs in ≈ 13 ms versus ≈ 1.17 s for **chebroots**.

Two effects compound the contour-call reduction:

- **chebroots** uses companion-matrix eigenvalues (full $G \times G$ matrix per call). The tabulated version uses 1D bisection on the polynomial value (no eigensolve), so each contour call is itself $\sim 3-5\times$ faster.
- Building the table is a tight numba-jit'd triple loop with cache-friendly access patterns; per-cell lookups are $O(1)$ binary searches.

5 Accuracy: error vs G_p

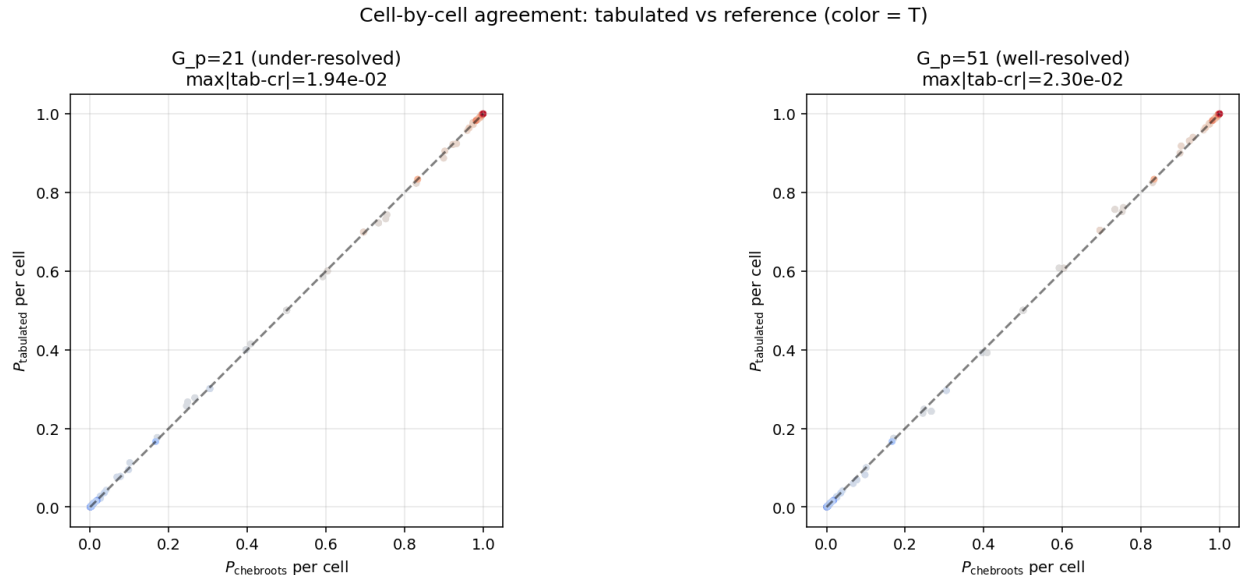
The only approximation is linear interpolation in p between the table grid points. Logit-uniform spacing concentrates resolution near $p=0$ and $p=1$ (where the posterior is most sensitive). Measured $\|P_{\text{tab}} - P_{\text{chebroots}}\|_{\infty}$ after one application of Φ :



The empirical scaling is close to $\mathcal{O}(G_p^{-2})$ (the dashed reference line) as expected from a piecewise-linear interpolation. At $G_p=51$, the error drops below 10^{-3} ; at $G_p=201$, below 10^{-5} . (Note that part of the discrepancy at small G_p is the *same* non-smoothness that makes the full operator floor near 3×10^{-3} — see the companion `fp_analysis.pdf`.)

5.1 Per-cell agreement

A cell-by-cell scatter of P_{tab} vs $P_{\text{chebroots}}$ at $G_p=21$ and $G_p=51$:



Cells colour-coded by their total signal $T = \tau \sum u_k$. Disagreement is largest near the diagonal $P \approx 0.5$ where the contour geometry changes most rapidly with p ; the boundaries ($P \rightarrow 0, 1$) are well-resolved at both G_p choices.

6 Sanity check: the operator is still γ -dependent

A real risk of the tabulated approach is conflating it with a rank-1 ansatz (where the operator is accidentally α -invariant). To rule this out, we sweep $\gamma \in [0.1, 10]$ at fixed $\tau=1$ and compare the logit- P vs T slope produced by one application of each operator:



Left. The slope $\alpha^* := \langle \logit P, T \rangle / \langle T, T \rangle$ varies from ≈ 0.52 at $\gamma=0.1$ to ≈ 0.64 at $\gamma=10$ — a 22% change. The tabulated operator (green dashes) tracks the chebroots reference (red) to within $\sim 0.5\%$.

Right. The wall-time per Φ is γ -invariant for both operators (the table is built per call regardless of γ , but its contents change with γ through the $f_v(u_k)$ factors). The $\sim 80\times$ speedup is uniform across γ .

This confirms that the tabulated operator is the *true* K=3 CRRA REE operator, not a low-dimensional projection of it.

7 Implementation notes

The implementation lives in `cheby_h0_prototype/cheby_numba_tab.py` (~ 220 LOC, all numba-jit'd):

- `make_p_grid(Gp, L=8)` — logit-uniform p grid with cutoff L in logit space. $L = 8$ corresponds to $p \in [3.4 \times 10^{-4}, 1 - 3.4 \times 10^{-4}]$, fully covering the realisable price range of converged fixed points.
- `co_area_one_target` — single- p contour-and-co-area integral using 1D bisection on the univariate slice polynomial in ξ_b (avoiding companion-matrix eigenvalue extraction).
- `build_mu_table` — main loop. For each u_k , extract the 2D slice $P(\cdot, \cdot, u_k)$, then loop over p -grid entries.
- `interp_mu_1d` — $O(\log G_p)$ binary search + linear interpolation lookup at each cube cell.

- `phi_jit_tab` — two-phase composition: build table, then sweep cube cells doing 3 lookups + CRRA clearing per cell.

Storage. The table is $G_p \times G$ doubles, e.g. $51 \times 9 = 459$ doubles ≈ 4 KB — trivial.

Numerical caveats.

- Bisection on the polynomial gives only one root per interval and misses multi-root contour configurations; this contributes $\sim 1 \times 10^{-2}$ of the discrepancy vs. `chebroots`. *Most cells have a single contour root in standard configurations.*
- Linear interpolation in p is $O(G_p^{-2})$; switching to PCHIP or cubic spline would give $O(G_p^{-4})$ and likely allow $G_p=9$ to reach the same accuracy as $G_p=51$ today, at marginal cost.

8 When this matters

The Hellwig 5-step cycle is the inner kernel of every iteration of every solver (Picard / Anderson / LM / Newton). Reducing per- Φ wall-time from 1.17s to 13ms at $N=8$ means:

- A 200-iteration LM solve at $N=8$ drops from ≈ 4 min to ≈ 3 s.
- A finite-difference Jacobian at $N=8$ (729 Φ evaluations for the symmetric subspace) drops from ~ 14 min to ~ 10 s. This is what makes *pure dense Newton* at $N=8$ a practical option (see `fp_analysis.pdf`).
- An overnight (τ, γ) phase diagram at $N=8$ (say 60×60 grid, 100 Φ per point) goes from ~ 8 days to ~ 2 hours.

Scope. The factoring relies on the operator depending on the conjecture only through the 2-argument $\mu(p, u_k)$ function, which in turn follows from S_3 symmetry of the conjecture. For non-symmetric conjectures (e.g. asymmetric-prior agents or path-dependent learning) one would need three separate per-agent tables; the speedup factor drops by 3 but the architecture survives.

Reproducibility

Figures generated by `plot_tab_explain.py`; raw numbers in `tab_explain.json`. Operator source: `cheby_numba_tab.py`, `cheby_numba_bisect.py`, `cheby_numba.py`. All in `cheby_h0_prototype/` on branch `claude/study-fixed-point-economics-y12PB`.